Topological Sort

AlgoExpert · Hard · Famous Algorithms

The problem

- Jobs with [prereq, job] dependencies; return a valid order or [] if there is a cycle (Kahn's algorithm).

Intuition

- Kahn's algorithm: repeatedly take jobs with no remaining prerequisites.

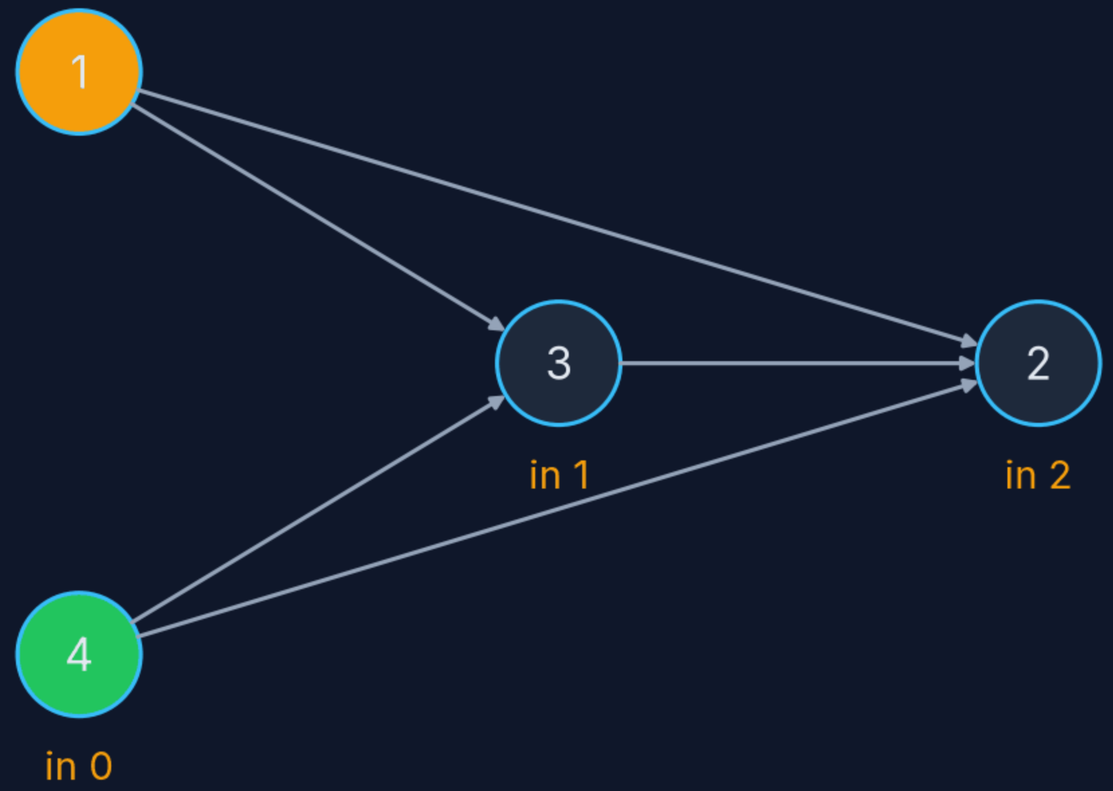
Brute force vs optimal

- Repeatedly scan for a job with no prerequisites → $O(V^2)$
- Kahn's algorithm with in-degrees → $O(V + E)$
- DFS post-order also works

Approach

- 1. Count in-degrees
- 2. Queue jobs with zero
- 3. Pop, append, and decrement dependents
- 4. If not all jobs appear, there's a cycle

Worked example

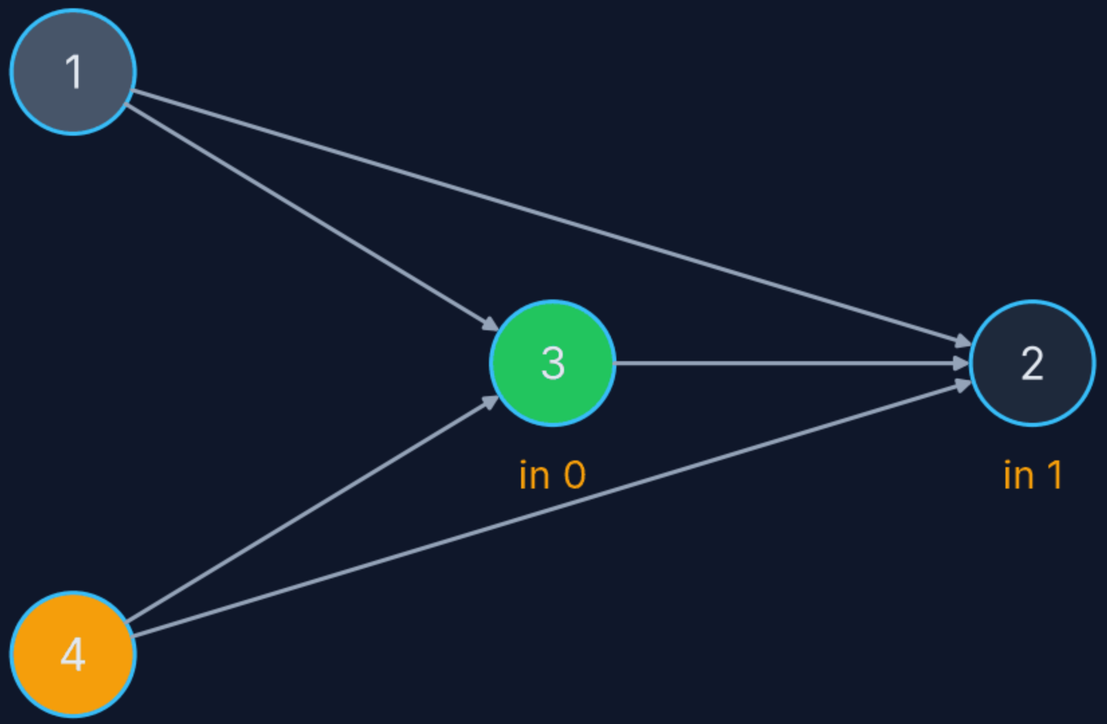



```
order: [1]

queue: [4]
```

Take 1

Worked example

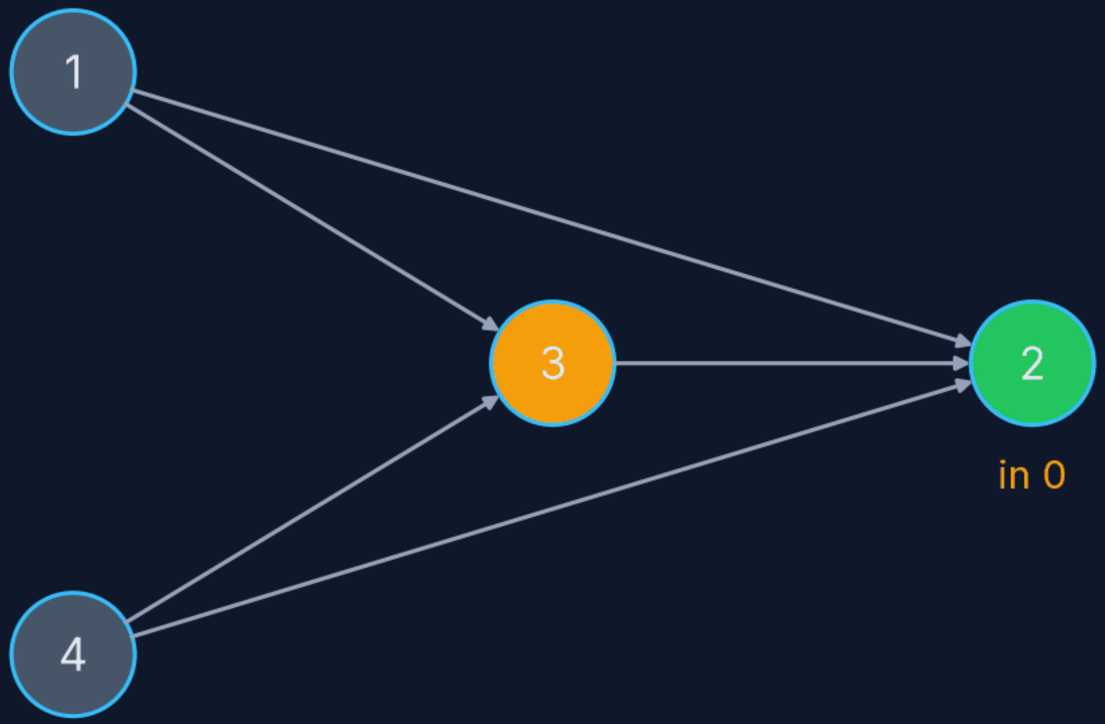



```
order: [1, 4]

queue: [3]
```

Take 4; [3] now free

Worked example

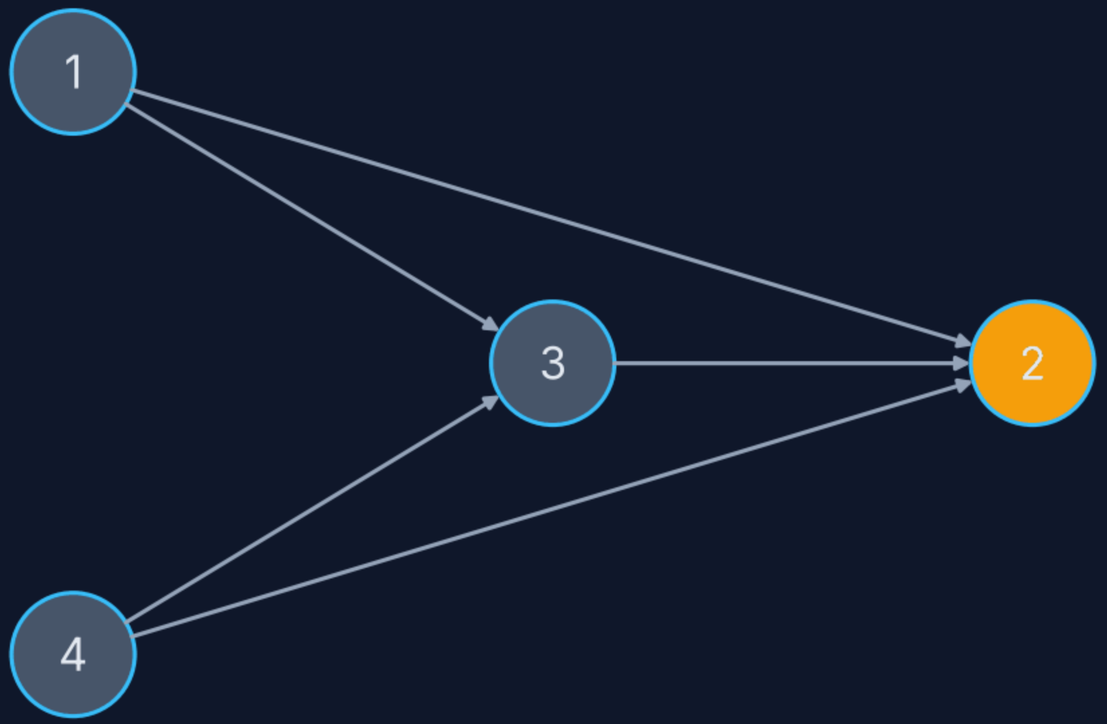



```
order: [1, 4, 3]

queue: [2]
```

Take 3; [2] now free

Worked example




```
order: [1, 4, 3, 2]

queue: []
```

Take 2

Java solution

```java
public class TopologicalSort {
    public static List<Integer> topologicalSort(List<Integer> jobs, List<Integer[]> deps) {
        Map<Integer, List<Integer>> g = new HashMap<>();
        Map<Integer, Integer> indeg = new HashMap<>();
        for (int j : jobs) { g.put(j, new ArrayList<>()); indeg.put(j, 0); }
        for (Integer[] d : deps) { g.get(d[0]).add(d[1]); indeg.merge(d[1], 1, Integer::sum); }
        ArrayDeque<Integer> q = new ArrayDeque<>();
        for (int j : jobs) if (indeg.get(j) == 0) q.add(j);
        List<Integer> out = new ArrayList<>();
        while (!q.isEmpty()) {
            int j = q.poll();
            out.add(j);
            for (int k : g.get(j)) if (indeg.merge(k, -1, Integer::sum) == 0) q.add(k);
        }
        return out.size() == jobs.size() ? out : new ArrayList<>();
    }
}
```

Complexity

- Time: O(j + d)
- Space: O(j + d)

Edge cases & pitfalls

- Cycle → fewer jobs output → []
- Multiple valid orders
- Isolated jobs

Interview tips & follow-ups

- Mention DFS-based alternative and cycle detection

Recap

- Kahn's algorithm: repeatedly take jobs with no remaining prerequisites.
- Complexity: O(j + d)